

**** Cell 7: Graph Retrieval (Multi-hop) ****

What This Code Does (Big Picture)

Query → Find nodes → Traverse graph → Collect relationships → Return context

👉 This is called:

Multi-hop Graph Retrieval 🔥

FULL CODE

```
def graph_retrieval(graph, query, max_hops=2, top_k=10):
```

👉 Inputs:

Parameter Meaning

graph Your knowledge graph

query User question

max_hops How far to traverse

top_k How many results to return

Step-by-Step Explanation

STEP 1: Normalize Query

```
query = query.lower()
```

👉 Converts:

"Fuel System" → "fuel system"

👉 Helps matching

STEP 2: Create Result Storage

```
results = []
```

👉 Will store:

```
(score, "node --relation--> node")
```

**** Cell 7: Graph Retrieval (Multi-hop) ****

STEP 3: Find Matching Nodes

```
for node in graph.nodes():  
    if node.lower() in query:
```

👉 This checks:

Does query contain this node?

Example:

Query:

"How fuel system works with combustion?"

👉 Matches:

Fuel System 

Combustion 

STEP 4: MULTI-HOP TRAVERSAL (CORE)

```
paths = nx.single_source_shortest_path_length(  
    graph, node, cutoff=max_hops  
)
```

👉 This means:

Start from node

Explore neighbors up to 2 steps (hops)

Example:

Fuel System → Fuel → Combustion

👉 Hop levels:

Hop Meaning

0 Fuel System

1 Fuel

2 Combustion

**** Cell 7: Graph Retrieval (Multi-hop) ****

● STEP 5: Traverse Nodes

for target in paths:

👉 target = each node reached

● STEP 6: Get Neighbors (Edges)

for neighbor in graph.neighbors(target):

👉 Finds:

Connections from target node

Example:

Combustion → Heat

Combustion → Turbine

● STEP 7: Get Relation

rel = graph[target][neighbor]['relation']

👉 Gets:

"produces", "drives", etc.

● STEP 8: Get PageRank Score

score = graph.nodes[neighbor].get('pagerank', 0)

👉 Importance of node:

High score = more important

● STEP 9: Store Result

results.append((score, f"{target} --{rel}--> {neighbor}"))

👉 Example:

**** Cell 7: Graph Retrieval (Multi-hop) ****

(0.25, "Combustion --produces--> Heat")

STEP 10: Sort by Importance

```
results = sorted(results, key=lambda x: x[0], reverse=True)
```

👉 Highest score first 🔥

STEP 11: Return Top Results

```
return "\n".join([r[1] for r in results[:top_k]])
```

👉 Only top 10 results

FINAL OUTPUT (Context)

Fuel System --supplies--> Fuel
Combustion --uses--> Fuel
Combustion --produces--> Heat
Combustion --generates--> High Pressure Gases
Turbine --drives--> Shaft

TEST QUERY FLOW

query = "How fuel system works with combustion?"

Step-by-step:

1. Match nodes:

- Fuel System
- Combustion

2. Traverse graph:

Fuel System → Fuel → Combustion → Heat

3. Extract edges:

- Fuel System --supplies--> Fuel
- Combustion --uses--> Fuel
- Combustion --produces--> Heat

**** Cell 7: Graph Retrieval (Multi-hop) ****

4. Rank using PageRank

5. Return top results

WHY THIS IS POWERFUL

Normal RAG

Just finds similar text

Your Graph RAG

Finds connected knowledge across documents

Multi-Hop Visual

Fuel System



Fuel



Combustion



Heat Turbine

Key Insight

Retriever = Brain of Graph RAG

One-Line Summary

This function retrieves relevant knowledge by matching query terms with graph nodes, performing multi-hop traversal to explore connected nodes, and returning the most important relationships ranked using PageRank.

What Happens Next

Retriever output → goes to LLM → final answer

**** Cell 7: Graph Retrieval (Multi-hop) ****

YOU NOW UNDERSTAND

Graph Construction 

PageRank 

Retriever (MOST IMPORTANT) 

What This Code Does (Big Picture)

Graph → Retriever → Context → LLM → Final Answer

 This function connects:

Your Graph (brain) + LLM (speaker)

Function Start

```
def graph_rag_answer(graph, query):
```

 Inputs:

Input Meaning

graph Your knowledge graph

query User question

STEP 1: Retrieve Context (MOST IMPORTANT)

```
context = graph_retrieval(graph, query)
```

 This calls your **retriever**

What happens here:

Query → Graph traversal → Multi-hop → Context

Example:

**** Cell 7: Graph Retrieval (Multi-hop) ****

Query:

How fuel system works with combustion?

Context returned:

Fuel System --supplies--> Fuel

Combustion --uses--> Fuel

Combustion --produces--> Heat

Print Context

```
print("\n🔍 Retrieved Context:\n")  
print(context)
```

👉 This is for debugging (very useful)

🔥 STEP 2: Create Prompt (VERY IMPORTANT)

```
prompt = f"""  
You are an expert system engineer.  
...  
"""
```

👉 You are preparing input for LLM

Prompt Structure

● Role

You are an expert system engineer.

👉 Guides LLM behavior

● Context

Context:
{context}

👉 This is your **Graph knowledge**

**** Cell 7: Graph Retrieval (Multi-hop) ****

Question

Question:

{query}

Instruction

Answer clearly and concisely

Why This is IMPORTANT

LLM does NOT see full documents ❌

LLM only sees Graph context ✅

👉 This makes it:

Efficient + accurate + explainable

STEP 3: Call LLM

```
response = client.chat.completions.create(
```

👉 This sends prompt to:

Azure OpenAI (your LLM)

Parameters Explained

model

model=DEPLOYMENT_NAME

👉 Your Azure deployed model

messages

```
messages=[  
    {"role": "system", "content": "..."},  
    {"role": "user", "content": prompt}  
]
```


**** Cell 7: Graph Retrieval (Multi-hop) ****

👉 Standard chat format

● **temperature**

temperature=0.3

👉 Controls randomness:

Value Behavior

0.0 very strict

0.3 balanced ✅

1.0 creative

🔥 **STEP 4: Extract Answer**

answer = response.choices[0].message.content

👉 Gets final response text

🔥 **STEP 5: Return Answer**

return answer

🧠 **FINAL FLOW**

User Query

↓

graph_retrieval()

↓

Context (triples)

↓

Prompt creation

↓

LLM

↓

Final Answer

**** Cell 7: Graph Retrieval (Multi-hop) ****

REAL EXAMPLE

Input:

How fuel system works with combustion?

Context:

Fuel System → Fuel → Combustion → Heat

Output:

The fuel system supplies fuel to the combustion process, where it is combined with air to produce heat and high-pressure gases.

Why This is Powerful

Normal LLM

Uses general knowledge

Your System

Uses YOUR graph knowledge

Key Insight

Retriever = Brain

LLM = Voice

One-Line Summary

This function retrieves relevant knowledge from the graph, constructs a prompt using that context, and uses an LLM to generate a final answer based only on the retrieved graph information.

You Have Completed FULL PIPELINE

PDF → Text → Triples → Graph → PageRank → Retriever → LLM → Answer

**** Cell 7: Graph Retrieval (Multi-hop) ****

🔥 **YOU BUILT**

🔥 COMPLETE GRAPH RAG SYSTEM 🔥